EDP-Aware Mapping of LLM Computation Blocks on Multicore Architectures

1st Anmol Gupta
Computer Science & Automation
Indian Institute of Science
Bengaluru, India
anmolgupta@iisc.ac.in

1st Aditya Dhodade
Computer Science & Automation
Indian Institute of Science
Bengaluru, India
adityand@iisc.ac.in

1st Pratika Chavan
Computer Science & Automation
Indian Institute of Science
Bengaluru, India
pratikac@iisc.ac.in

***Abstract*—Large language models (LLMs) are increasingly being deployed on local platforms, making Energy Delay Product (EDP) efficient inference an important objective. Modern heterogeneous CPUs provide an opportunity to improve the EDP by mapping different types of computations to suitable cores. However, LLM inference is inherently diverse because the operations such as matrix multiplications in feed forward layers are highly compute intensive while computation blocks like normalization and softmax are primarily memory bound. Despite this, existing CPU based inference frameworks such as llama.cpp largely rely on uniform execution strategies, treating all operations similarly and failing to make adapt to this diversity, which leads to suboptimal performance and energy efficiency. In this work, we propose a KV cache aware dynamic mapping strategy within the GGML backend to better align workload characteristics with hardware capabilities. Our approach first classifies transformer operations into compute bound and memory bound categories and assigns them to suitable cores accordingly. In addition to this static classification, we explicitly consider the dynamic behavior of attention operations during inference. As the KV cache grows with increasing sequence length, attention gradually shifts from being compute bound to memory bound. To capture this transition, our scheduler identifies a shift point based on runtime behavior and remaps attention operations to more appropriate cores beyond this point. This design aims to adapt both to the diversity of LLM workloads and to their evolving execution characteristics over time. However, our experimental evaluation shows that the benefits of this dynamic strategy are limited by system level overheads. Frequent scheduling decisions, repeated thread affinity changes and thread migration across core groups introduce additional latency, disrupt cache locality and increase memory traffic. As a result, these overheads can outweigh the potential energy savings, leading to degraded performance and higher overall EDP compared to baseline execution. These findings highlight that while KV cache aware dynamic mapping captures important workload phase changes, its effectiveness depends critically on minimizing runtime overheads and maintaining data locality which eventually increases the latency of the LLM inference.**

***Index Terms*—LLM Inference, Heterogeneous system, Energy-Delay Product (EDP), CPU Scheduling, Performance-Efficiency Cores, Dynamic Scheduling, Static Mapping, Energy Optimization**

I. Introduction

Large language models (LLMs) are increasingly being executed on local devices, making EDP efficient inference a critical concern. Modern heterogeneous CPUs provide an opportunity to optimize the Energy Delay Product (EDP) by mapping different types of computations to suitable cores. However, LLM inference workloads are diverse. Operations such as matrix multiplications in feed forward and attention layers are highly compute intensive while components like normalization and softmax are primarily memory bound. This difference can be found through arithmetic intensity which measures the ratio of computation to data movement operations. The blocks with high intensity are compute bound whereas those with low intensity are memory bound. In transformer architectures, this classification appears naturally at the operator level like feed forward layers dominated by large matrix multiplications are consistently compute bound due to high data reuse and cache efficiency while normalization and softmax involve repeated data access with limited computation whcih make them memory bound. Attention mechanisms exhibit dynamic behavior in generation. When the KV cache is small and fits in cache, attention behaves as compute-bound but as the cache grows beyond capacity, increased memory accesses and cache misses cause it to shift toward a memory bound . Despite this variability across operations and over time, most CPU based inference frameworks, including llama.cpp, adopt uniform execution strategies that treat all operations similarly, leading to inefficient hardware utilization and suboptimal performance and energy efficiency.

In this work, we introduce a KV cache aware dynamic mapping strategy within the GGML backend of llama.cpp to better align computational workloads with the capabilities of heterogeneous CPU architectures. Our approach begins with a static classification of transformer operations into compute bound and memory bound categories, followed by mapping them to suitable core types. Compute-intensive operations such as feed-forward matrix multiplications and early-stage attention, are assigned to performance cores (P-cores), while memory-bound operations including normalization and softmax are scheduled on efficiency cores (E-cores). In addition to this classification, we explicitly account for the dynamic behavior of attention operations during inference. As inference progresses and the KV cache grows with increasing sequence length, attention gradually shifts from being compute-bound to memory-bound. Initially, when the KV cache is small and fits within the processor cache hierarchy, attention behaves as a compute intensive operation. However, as the cache

exceeds the capacity of the last-level cache, memory access latency becomes the dominant factor which to a shift in its execution characteristics. To capture this transition, our scheduler monitors runtime behavior using pertoken latency as a lightweight proxy for memory pressure. When a sustained increase in latency is observed, the scheduler identifies a shift point and dynamically remaps attention operations to more suitable cores i.e. moving them from P-cores to E-cores beyond this point—while continuing to execute feed-forward layers on P-cores.

We evaluate the effectiveness of this KV cache-aware mapping strategy on a heterogeneous CPU system comprising performance and efficiency cores across five models. EDP is calculated for this. The baseline for comparison is the default configuration of llama.cpp, which uses no explicit core pinning and relies on standard frequency scaling mechanisms. Our experimental results show that the benefits of this dynamic mapping approach are constrained by system level overheads. Frequent scheduling decisions, repeated thread affinity updates and thread migration across different core types introduce additional latency. These operations also disrupt cache locality. As a result, these overheads outweigh the potential benefits of improved workload core alignment, leading to degraded performance. In particular, the increase in latency directly causes the overall Energy Delay Product (EDP) to be higher than the baseline.

These findings highlight a tradeoff. While KV cache-aware dynamic mapping successfully captures the evolving nature of workload characteristics practical effectiveness depends on minimizing runtime overheads and preserving data locality. On current heterogeneous CPU systems, the cost of dynamic adaptation outweighs the gains, limiting the overall efficiency improvements of this approach.

II. Related Work

A. Workload Characterization for Transformer Models

Understanding the distinct computational properties of different transformer operations is foundational for effective resource allocation. The key concept here is *arithmetic intensity*, the ratio of arithmetic operations to data movement (bits or bytes accessed). Operations with high intensity are compute-bound, benefiting from high-frequency cores, while those with low intensity are memory-bound, where energy efficiency becomes paramount. This principle is formally captured by the Roofline Model [1], which ties together floating-point performance, operational intensity, and memory performance. The Energy Roofline Model [2] extends this to explicitly model energy–performance trade-offs. Building on these principles, characterization studies of transformer models have shown that feed-forward network (FFN) layers, dominated by large matrix multiplications (GEMMs), exhibit consistently high arithmetic intensity, making them compute-bound across all phases of inference. In contrast, normalization (LayerNorm, RMSNorm) and softmax operations involve repeated data access with limited computation, rendering them inherently memory-bound [3]. A crucial and more recent insight is the non-stationary behavior of the attention mechanism during autoregressive generation. As the Key-Value (KV) cache grows linearly with sequence length, the computational characteristics of attention fundamentally shift. For short sequences where the KV cache fits in the processor's cache hierarchy, attention is compute-bound [16]. However, as the sequence length increases and the KV cache exceeds last-level cache capacity, memory access latency dominates, and attention transitions into a memory-bound regime. While this has been recognized in work on memory-efficient attention such as FlashAttention [14] and PagedAttention [16], as well as streaming LLM architectures [6], existing scheduling strategies largely treat attention uniformly, missing this phase-specific optimization opportunity. This makes our work one of the first to propose a scheduling policy that explicitly accounts for KV cache growth.

B. Hardware–Software Co-Optimization for CPU Inference

Substantial effort has gone into co-designing algorithms and hardware to optimize transformer inference on CPUs. This includes leveraging SIMD vectorization, exploiting cache hierarchies, and performing operator fusion to reduce overheads. Production frameworks, notably `llama.cpp` and its underlying GGML tensor library, have emerged as practical solutions, providing efficient CPU inference through optimized kernels and quantization (e.g., Q4_0, Q5_1) [7], [8]. However, these systems largely adopt homogeneous execution strategies, where the OS scheduler distributes threads uniformly across cores without considering the heterogeneous computational requirements of different transformer operations. This presents a clear opportunity, which our work addresses, to optimize inference by aligning operation types with core capabilities.

C. Energy-Aware Scheduling on Heterogeneous Processors

Modern heterogeneous CPUs, such as Intel's hybrid architecture (P-cores/E-cores) and ARM's big.LITTLE, provide a mix of core types optimized for performance and efficiency. Scheduling on these architectures has been an active research area. Simple approaches like thread pinning and favoring performance cores for select computations can improve energy efficiency, as they reduce scheduler overhead and maintain better cache locality. Using Linux's `isolcpus` kernel parameter for core isolation can reduce OS scheduling variability [9]. More sophisticated work has explored offline scheduling models to find energy-optimal configurations for LLM inference on heterogeneous systems, achieving high accuracy for runtime and energy predictions [10]. However, the application of these techniques for aligning operation types with core capabilities during LLM inference, especially while accounting for sequence-dependent phase changes, remains largely unexplored.

D. Overhead of Dynamic Scheduling and Thread Migration

While dynamic scheduling offers theoretical benefits, the practical overheads of implementation can be substantial. Key sources of overhead include cache and TLB invalidation,

system call overhead, and OS scheduler pressure. When a thread migrates between cores, it loses cache locality because L1 and L2 caches are not preserved, forcing the destination core to start with cold caches. This leads to capacity and conflict misses and increased DRAM access. Similarly, TLB entries can become invalid, increasing page walk latency. The system calls introduces latency. Thread migration has shown performance penalties due to increases in L2 cache misses [12]. To mitigate these costs, coarse grained scheduling has been proposed, where affinity changes happen only at detected phase boundaries rather than per operation [13]. Our KV cache aware strategy aligns with this concept by using a latency based phase detection trigger.

E. Sequence Dependent Behavior of Attention in LLM Inference

A key characteristic of autoregressive LLM inference is that the Key Value (KV) cache grows linearly with the generated sequence length. This growth fundamentally alters the computational properties of the attention mechanism over time. For short sequences, the KV cache fits within the last level cache (LLC), attention operations are dominated by matrix multiplications and exhibit high arithmetic intensity, making them compute bound. However, once the cache exceeds LLC capacity, memory access latency and bandwidth become the bottleneck and attention transitions to a memory bound phase. This non stationary behavior has been recognized in recent optimization efforts [14], [16], yet most scheduling frameworks treat attention uniformly throughout inference, missing opportunities for phase aware adaptation.

Prior work on phase detection in heterogeneous systems has explored lightweight methods to identify changes in workload behavior. Bhattacharjee et al. [13] proposed phase aware thread to core mapping for multicore processors, using hardware performance counters to detect phase transitions and migrate threads accordingly. However, their evaluation focused on general purpose workloads rather than the iterative, cache sensitive pattern of LLM generation. Similarly, Power et al. [11] studied coherence overheads in heterogeneous CPU GPU systems, highlighting how data movement across compute domains can dominate execution time. Offline scheduling models for energy optimal LLM execution [10] have been proposed, but they do not account for online phase changes. The cost of thread migration itself has been quantified for multi threaded applications [12], showing that frequent affinity changes can severely impact performance.

The specific challenge of KV cache growth in LLMs has been addressed in optimization works such as PagedAttention [16], which reduces memory fragmentation and FlashAttention [14], which improves IO awareness by tiling and recomputation to minimize memory reads/writes. More recent work, such as RocketKV [17], explicitly identifies the growing KV cache as burdening both memory bandwidth and capacity during decoding. However, these works focus on algorithmic and memory layout improvements rather than on exploiting the phase transition for dynamic core mapping on heterogeneous CPUs. To the best of our knowledge, no prior work has proposed or evaluated a runtime scheduling strategy that explicitly monitors KV cache pressure to remap attention operations to different core types. Our work fills this gap by implementing a lightweight latency jump detector that triggers a one time migration of attention from P cores to E cores after the compute bound to memory bound transition.

III. Methodology

A. Setup used for the experiment

All experiments are conducted on a heterogeneous system with an Intel® Core™ Ultra 5 115U processor. The system consists of a total of 10 CPU cores, which are divided into two groups based on hardware characteristics and operating frequency: performance cores (P-cores) comprising cores 0–3, and efficiency cores (E-cores) comprising cores 4–9. We used the llama.cpp framework as the inference engine for executing large language models (LLMs). The models evaluated in this work include Meta-Llama-3.1-8B-Instruct, Mistral-7B-Instruct-v0.2, Qwen2.5-7B-Instruct, Gemma-2-9B-IT, and Phi-3.5-mini-Instruct. Inference is performed using a fixed set of prompts across all models to ensure consistency. Additionally, the number of generated tokens is fixed at 128 for all runs, enabling fair comparison of latency and energy measurements across different experimental configurations.

B. Measurement methodology

Latency is measured as the difference in time between the start and end time of inference. Energy consumption is measured using Intel Running Average Power Limit (RAPL) counters.

Energy is recorded before and after inference and difference is calculated to get the total energy cosumption. The EDP is calculated by taking product of latency and energy.

C. KV Cache Aware Dynamic Task Allocation

To optimize inference on heterogeneous processors, we implemented dynamic task allocation within the GGML runtime. This routes compute bound operations to performance cores (P-cores) and memory bound operations to efficiency cores (E-cores). Specifically, we modified the computational graph evaluation loops within the `ggml-cpu.c` backend, allowing the executor to classify and route individual transformer operations immediately before dispatch.

This dynamic scheduling is based on a hybrid classification. Initially, tensor naming heuristics are applied by inspecting the internal identifiers of the graph nodes. Tensors associated with attention queries, keys, or feed-forward networks (e.g., `blk.0.attn_q.weight`) are classified as compute bound due to their reliance on dense matrix multiplications. Conversely, operations handling layer normalization, residual additions, or softmax transformations (e.g., `blk.0.attn_norm.weight`) are classified as memory bound. Based on this classification, the system uses the Linux `pthread_setaffinity_np()` system call to bind

compute-bound threads to P-cores (CPUs 0 to 3) and memory bound threads to E-cores (CPUs 4 to 9).

To reduce the context-switching overhead associated with requesting OS level affinity changes for every operation, our implementation utilizes thread-local states. The system queries this state and only performs an affinity switch when a thread transitions between compute bound and memory bound operations. Furthermore, feed forward network (FFN) layers, which rely on large matrix multiplications, are statically classified as compute bound and remain mapped to the P cores throughout the inference process.

While fixed rules work well for operations with predictable behaviors, attention mechanisms change dynamically. At the beginning of token generation, the Key-Value (KV) cache is relatively small. Because this minimal data fits easily inside the processor's Last Level Cache (LLC), attention operations run quickly as compute bound tasks. However, as the model generates more tokens, the KV cache grows. Once the cache becomes too large for the LLC to hold, the processor is forced to constantly fetch data from the slower main memory. This creates a bottleneck, causing the attention operations to naturally transition from being compute bound to memory bound.

To adapt to this change, the scheduler monitors memory pressure in real-time. Instead of using heavy hardware profiling tools that slow the system down, we simply track how long it takes to process each token. Specifically, the scheduler records the exact time right before and after the `ggml_compute_thread` function runs. By measuring this time difference for every generated token, we can accurately track how the growing cache affects the speed of the attention operations.

To make sure we do not accidentally trigger a shift due to random system delays, we smooth these measurements using a moving average. If L_t represents the `ggml_compute_thread` latency for the current token t, the moving average $\overline{L}_{t-1}$ over the previous $W = 5$ tokens is calculated as:

$$\overline{L}_{t-1} = \frac{1}{W} \sum_{i=1}^{W} L_{t-i} \tag{1}$$

The system detects a shift when the latency of the newest token jumps more than 25% above this moving average. This rule is expressed as:

$$\frac{L_t - \overline{L}_{t-1}}{\overline{L}_{t-1}} > 0.25 \tag{2}$$

Once this condition is met, the system recognizes that the attention mechanism has become memory bound. It then overrides the initial routing and moves all future attention operations to the energy efficient E cores.

IV. Results and Discussion

We evaluated the KV-cache aware dynamic mapping strategy against the default baseline configuration across five LLM models.

Tables I and II report the baseline and KV-cache aware results, respectively.

TABLE I
BASELINE PERFORMANCE (DEFAULT LLAMA.CPP). LATENCY IN SECONDS, ENERGY IN JOULES, EDP IN J·S.

Model	Latency	Energy	EDP
Meta-Llama-3.1-8B	68.72	537.5	36939.41
Mistral-7B	57.69	457.7	26400.41
Qwen2.5-7B	62.38	491.0	30624.31
Gemma-2-9B	84.84	690.3	58565.82
Phi-3.5-mini	24.37	355.1	8650.06

TABLE II
KV-CACHE AWARE MAPPING PERFORMANCE. LATENCY IN SECONDS, ENERGY IN JOULES, EDP IN J·S.

Model	Latency	Energy	EDP
Meta-Llama-3.1-8B	92.68	572.4	53054.57
Mistral-7B	75.86	482.9	36630.57
Qwen2.5-7B	80.88	525.4	42491.25
Gemma-2-9B	112.13	724.8	81260.08
Phi-3.5-mini	31.89	376.4	12001.96

A. Impact on Latency

For every model, KV cache aware mapping increases inference latency but the relative increase varies across models. Meta-Llama-3.1-8B shows the largest rise, from 68.72 seconds to 92.68 seconds, an increase of 34.9%. Mistral-7B increases by 31.5% (from 57.69 to 75.86 seconds), Qwen2.5-7B by 29.7% (from 62.38 to 80.88 seconds), Gemma-2-9B by 32.2% (from 84.84 to 112.13 seconds), and Phi-3.5-mini by 30.9% (from 24.37 to 31.89 seconds). The variation is because of the differences in model architecture and how aggressively the KV cache grows but the overall pattern is consistent, all models suffer a substantial latency penalty.

B. Energy Consumption

Energy also increases in the KV cache aware configuration, with the rise ranging from 5% to 7% across the models. Meta-Llama-3.1-8B consumes 572.4J compared to 537.5J in the baseline (a 6.5% increase). Mistral-7B increases by 5.5%, Qwen2.5-7B by 7.0%, Gemma-2-9B by 5.0%, and Phi-3.5-mini by 6.0%. This extra energy comes mainly from the overhead of frequent thread affinity changes, additional cache misses and extra DRAM traffic when threads migrate between core types. The energy increase is modest relative to the latency increase, so the overall EDP degradation is impacted primarily by the longer inference time.

C. Why Does KV-Cache Aware Mapping Increase Latency?

The proposed strategy introduces frequent thread affinity changes by invoking `pthread_setaffinity_np()`. Since each inference consists of multiple components such as attention layers, feed forward blocks and normalization steps, this leads to thousands of affinity updates per run. Although each individual call incurs only a small overhead (approximately 2–5 μs), the cumulative impact becomes significant

when combined with indirect effects such as scheduling delays. As a result, the overall latency increase is substantially higher than the theoretical cost of affinity calls alone.

Also, thread migration across heterogeneous cores disrupts cache locality. When a thread moves between P-cores and E-cores, its L1 and L2 cache contents are not preserved, forcing the destination core to start with cold caches. Consequently, frequently accessed data including model weights and KV cache entries must be reloaded from the shared last level cache or main memory. This repeated cache warm up increases memory access latency and directly contributes to longer inference times. The additional memory traffic also leads to a slight increase in overall energy consumption.

Frequent thread movement places additional pressure on the operating system scheduler. The Linux scheduler must repeatedly perform load balancing and context switching to satisfy changing affinity constraints. These operations interrupt execution continuity and introduce variability in scheduling decisions and threfore, increases latency. In contrast, the baseline configuration avoids such overheads by maintaining stable thread placement throughout execution.

D. Performance Tradeoffs in Dynamic Mapping

The static classification of transformer operations—putting feed-forward networks and attention on P-cores, and normalization, softmax, and other operations on E-cores—also had some overhead. The code had to check tensor names and set initial thread affinity. But this overhead was small because the assignments never changed during inference. The main idea was about attention. As the KV cache grows, attention becomes memory-bound. Moving it from P-cores to E-cores after that shift was supposed to save energy without hurting latency much, which would lower the Energy Delay Product. The results show the opposite behaviour, both latency and energy went up and EDP got worse. The reason is that the overhead from dynamic changes was much larger than any possible gain.

The system calls `pthread_setaffinity_np()` every time an attention block is reclassified after the shift point. A typical LLM has many attention layers and each generated token goes through all of them. So one inference can cause hundreds or even thousands of these calls. Each call only costs microseconds, but the total time adds up to seconds because of scheduling delays and cache misses. When a thread moves from a P-core to an E-core, it loses its L1 and L2 caches. The new core has cold caches and must reload model weights, KV cache entries, and other data from slower memory. This reloading makes latency longer and uses more energy. The extra energy from DRAM traffic cancels out any saving from running attention on Ecores. The operating system scheduler also has to do more work, causing extra delays.

Even though we change affinity only once per attention block, we still do it for every token after the shift. A better design would detect the shift once and then keep the same mapping for the rest of the run. But finding the exact shift point without adding more overhead is hard. Also, current heterogeneous CPUs have no fast way to move a thread with its cache data. There is no hardware to transfer hot cache lines from one core to another. So migration always starts with a cold cache, which is too slow for LLM inference.

V. Conclusion and Future Work

In this work, we proposed a KV cache aware dynamic mapping strategy for LLM inference on heterogeneous multicore CPUs which aims to reduce the Energy Delay Product (EDP) by aligning compute bound operations with performance cores and memory bound operations with efficiency cores. Our approach statically classified feed forward networks (P cores) and normalization/softmax (E cores) while dynamically reclassifying attention based on KV cache growth using per token latency monitoring. Although the underlying phase transition is real and measurable, experimental evaluation across five LLM models revealed that dynamic mapping increased both latency (29–35%) and energy (5–7%), leading to a 40–50% increment in EDP compared to the baseline. The primary reasons are system level overheads like frequent thread affinity changes, cache locality disruption during core migration and additional OS scheduler pressure. These overheads outweigh expected improvements demonstrating that on current heterogeneous systems, the cost of changing placement dominates the benefit of optimal placement.

Future work can focus on improving data locality in attention to reduce the memory bottleneck without introducing scheduling overheads. In particular, KV cache aware tiling can be explored to restructure attention computation so that smaller blocks of the KV cache are processed at a time, improving cache reuse and reducing DRAM accesses. Complementing this, software prefetching techniques can be used to proactively load KV cache data into cache before it is needed, thereby reducing stall time when attention becomes memory bound. These approaches directly target poor memory locality, while avoiding the costs associated with thread migration and dynamic affinity changes which makes them practical and effective directions for improving EDP in LLM inference.

Additionally, future work can explore dynamic voltage and frequency scaling (DVFS) instead of relying only on static governor settings. Core frequencies can be adjusted based on computation type, with higher frequencies for compute bound operations and lower frequencies for memory bound ones. This adaptive scaling has the potential to further reduce energy consumption and improve overall EDP.

References

[1] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009. https://dl.acm.org/doi/10.1145/1498765.1498785

[2] J. Choi et al., "A Roofline Model of Energy," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, 2013. https://ieeexplore.ieee.org/document/6656114

[3] T. Husom et al., "Phase-wise Characterization of LLM Inference," *arXiv preprint arXiv:2512.03024*, 2024. https://arxiv.org/abs/2512.03024

[4] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. ACM Symp. Operating Systems Principles (SOSP)*, 2023. https://arxiv.org/abs/2309.06180

[5] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. https://openreview.net/forum?id=FwGx2h7wZ1
[6] G. Xiao et al., “Efficient Streaming Language Models with Attention Sinks,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. https://arxiv.org/abs/2309.17453
[7] G. Gerganov, “GGML: A Tensor Library for Machine Learning,” *GitHub*. https://github.com/ggerganov/ggml
[8] “llama.cpp: LLM Inference in C/C++,” *GitHub*. https://github.com/ggerganov/llama.cpp
[9] “Overcoming OS-level Challenges for Low-Latency Inference,” *Eunomia*, Feb. 2025. https://eunomia.dev/blog/2025/02/18/os-level-challenges/
[10] “Offline Scheduling Models for Energy-Optimal LLM Execution,” *arXiv preprint arXiv:2407.04014*, 2024. https://arxiv.org/abs/2407.04014
[11] J. Power et al., “Heterogeneous System Coherence for Integrated CPU-GPU Systems,” in *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2013. https://ieeexplore.ieee.org/document/6676645
[12] “Understanding the Cost of Thread Migration for Multi-Threaded Java Applications,” *ISPASS*, 2009. https://ieeexplore.ieee.org/document/4919826
[13] A. Bhattacharjee et al., “Phase-Aware Thread-to-Core Mapping for Heterogeneous Multicore Processors,” in *Proc. IEEE Int. Conf. Parallel Distrib. Syst. (ICPADS)*, 2019. https://ieeexplore.ieee.org/document/8976585
[14] T. Dao et al., ”FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” *arXiv preprint arXiv:2205.14135*, 2022. https://arxiv.org/abs/2205.14135
[15] Z. Zhang et al., ”H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models,” *arXiv preprint arXiv:2306.14048*, 2023. https://arxiv.org/abs/2306.14048
[16] W. Kwon et al., ”PagedAttention: Efficient LLM Serving through Attention with Virtual Pages,” *arXiv preprint arXiv:2309.06180*, 2023. https://arxiv.org/abs/2309.06180
[17] Z. Wang et al., ”RocketKV: Accelerating Long-Context LLM Inference via Two-Stage KV Cache Compression,” *arXiv preprint arXiv:2411.04464*, 2024. https://arxiv.org/abs/2411.04464